

FENTON NONLINEAR WAVE SUITE

Theoretical framework, numerical formulation, solver architecture, and usage manual

1. Scope and purpose

This repository is an engineering and research toolchain for **steady, two-dimensional, periodic gravity waves of finite amplitude**. Its central purpose is to compute the wavelength, free-surface geometry, kinematics, and integral invariants of nonlinear waves in finite depth, optionally in the presence of a prescribed **Eulerian current**. The core physical model follows the **stream-function / Fourier-approximation formulation** implementation philosophy of Fenton's steady-wave program lineage (Rienecker and Fenton, 1981; Fenton and Rienecker, 1982; Fenton, 1988; Fenton, 1990; Fenton, 1999a).

The suite contains a main GUI interactive solver, a monolithic C++ implementation, reusable wavelength APIs, batch table generation tools, multiple surrogate-model builders, Excel/VBA ports, and nomogram generators. The common scientific task behind all of them is the same:

given wave height H , wave period T , water depth d , and current U_c , determine a dynamically consistent nonlinear wavelength L and, when the full solver is used, reconstruct the entire steady wave field.

Two project-specific explicit approximate formulas are also documented in this manual. The first is the genetically evolved baseline-plus-correction formula. The second is an effective transformed-variable formula, in which modified effective period and effective depth are passed through a linear dispersion solve. Both should be understood as compact approximate surrogates layered on top of physically meaningful wavelength structure rather than as replacements for the fully nonlinear Fourier / stream-function solver.

2. What this suite actually contains

The project is not a single calculator. It is a layered computational ecosystem composed of a **reference fully nonlinear solver** and several **operational derivatives** built around it.

2.1 Core solver and callable kernel

`fenton_gui.py` and `fenton_gui.exe`

Main GUI interactive nonlinear-wave application. It runs the Fenton stream-function solver for a no-current case and a current case, then formats an engineering report.

`function.py`

Standalone wavelength API and CLI. It exposes $L(H, T, d, U_c)$ for reuse by the rest of the toolchain.

`fourier.cpp`

Single-translation-unit modern C++ replica implementation of Fenton's steady-wave solver. It reads `data.dat` or CLI input, solves the nonlinear problem, and writes `solution.res`, `surface.res`, and `flowfield.res`.

2.2 Batch-processing and tabulation tools

`tables.py`

Batch table generator. It builds wavelength tables by repeatedly calling the external Fenton wavelength function.

2.3 Reduced-order surrogate builders

`pade.py`

Piecewise Padé/rational surrogate builder. It fits shallow-, intermediate-, and deep-water rational corrections and emits standalone Python and VBA evaluators.

`neural.py`

Global neural surrogate builder. It selects features, trains a tanh hidden-layer network, and emits standalone Python and VBA evaluators.

`genetic.py`

Symbolic-regression system. It uses gene expression programming to evolve an explicit correction factor over a linear baseline.

2.4 Nomogram and graphical tools

`nomogram_plots.py`

Nomogram/report generator. It creates printable nomograms for a compact explicit surrogate.

`nomogen_plots.py`

Nomogen-based nomogram generator. It uses the same surrogate kernel but a different graphics engine and layout logic.

2.5 Excel and VBA implementations

`linear.bas`

Excel/VBA linear baseline solver. It computes the Doppler-shifted Airy wavelength through Newton iteration.

`fenton.bas`

Excel/VBA nonlinear Fourier solver. It ports the continuation / Newton / SVD solver into VBA.

`pade.bas`

Excel/VBA Padé evaluator. It implements the emitted regime-based rational surrogate.

`neural.bas`

Excel/VBA neural evaluator. It implements the emitted tanh-network surrogate.

`genetic.bas`

Excel/VBA symbolic-regression evaluator. It implements the explicit GEP formula in Excel.

`effective.bas`

Excel/VBA effective transformed-variable evaluator. It computes an effective period and an effective depth from (H, T, d, U_c) , then solves the linear finite-depth dispersion relation using those transformed variables.

wavelength.xlsm

Excel workbook front-end. It is the operational spreadsheet wrapper exposing the VBA implementations.

The central physical reference is the **Fenton Fourier / stream-function solver**. Every other component falls into one of four categories:

1. **Direct nonlinear solvers:** `fenton_gui.py`, `fourier.cpp`, `fenton.bas`.
 2. **Exact or reference callable wavelength kernels:** `function.py`.
 3. **Reduced-order surrogates and explicit approximate formulas:** `pade.py`, `neural.py`, `genetic.py`, `genetic.bas`, `effective.bas`, and the other emitted VBA modules.
 4. **Presentation and tabulation tools:** tables, nomogram generators, and the workbook interface.
-

3. Physical problem definition

The entire suite is based on the canonical problem of a **steady progressive surface gravity wave** travelling over a horizontal bed in an inviscid, incompressible fluid. The wave is assumed to be two-dimensional and periodic. Surface tension is neglected. The fluid density is constant, and the flow is irrotational unless otherwise stated.

3.1 Geometry and coordinates

Let

- x be the horizontal coordinate,
- y be the vertical coordinate, positive upward,
- $y = -d$ be the impermeable flat bed,
- $y = \eta(x, t)$ be the free surface,
- L be the wavelength,
- H be the wave height,
- $k = 2\pi/L$ be the wavenumber,
- T be the wave period,
- $\omega = 2\pi/T$ be the angular frequency.

A steady progressive wave of phase speed c can be represented as a travelling form

$$\eta(x, t) = \eta(\xi), \quad \xi = x - ct.$$

In the frame moving with the wave, the solution becomes time-independent. This moving-frame formulation is essential because the fully nonlinear free-boundary problem then reduces to a **steady boundary-value problem** rather than an explicitly time-dependent one.

3.2 Kinematics and current definition

The horizontal and vertical velocities are

$$u = \frac{\partial \phi}{\partial x}, \quad v = \frac{\partial \phi}{\partial y},$$

where ϕ is the velocity potential. Because the flow is incompressible,

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0.$$

Because the flow is irrotational,

$$\frac{\partial v}{\partial x} - \frac{\partial u}{\partial y} = 0.$$

These conditions imply that both the velocity potential ϕ and the stream function ψ are harmonic. The current parameter used throughout the suite is an **Eulerian current** U_c , meaning a depth-averaged or otherwise prescribed ambient current component aligned with the direction of propagation. The presence of current modifies the Doppler relation between frequency and wavelength and therefore changes the nonlinear solution branch for given (H, T, d) .

3.3 Why the wavelength is nonlinear

For infinitesimal-amplitude waves in the absence of current, linear theory gives the classical dispersion relation

$$\omega^2 = gk \tanh(kd).$$

When wave amplitude is no longer negligible, the wavelength is no longer determined by linear dispersion alone. The phase speed, pressure field, crest-trough asymmetry, and free-surface curvature all modify the dynamically consistent wave state. For a given (H, T, d, U_c) , the true wavelength is therefore the result of a **nonlinear free-boundary solution**, not merely a linear dispersion evaluation. This is the central reason the suite exists: it computes or approximates

$$L = \mathcal{F}(H, T, d, U_c)$$

from a nonlinear theory rather than from the Airy approximation.

4. Governing equations of nonlinear steady wave motion

The theoretical foundation is the exact inviscid free-surface formulation.

4.1 Laplace equation in the fluid domain

Since the flow is incompressible and irrotational, the velocity potential satisfies

$$\nabla^2 \phi = \frac{\partial^2 \phi}{\partial x^2} + \frac{\partial^2 \phi}{\partial y^2} = 0$$

throughout the fluid domain

$$-d \leq y \leq \eta(x, t).$$

Equivalently, the stream function ψ satisfies the same harmonic condition.

4.2 Bed impermeability

At the flat bed, the normal velocity must be zero:

$$v(x, -d, t) = 0.$$

In terms of the stream function, the bed is a streamline. This is one reason the stream-function formulation is especially attractive: impermeable boundaries are encoded very naturally.

4.3 Free-surface kinematic condition

A particle on the free surface remains on the free surface. In Eulerian form,

$$\frac{\partial \eta}{\partial t} + u \frac{\partial \eta}{\partial x} = v \quad \text{at } y = \eta(x, t).$$

In the frame moving with speed c , where the flow becomes steady, the horizontal velocity is replaced by its relative value $(u - c)$, and the kinematic condition becomes

$$(u - c)\eta_x = v \quad \text{on } y = \eta(x).$$

This expresses the fact that the free surface is itself a streamline in the moving frame.

4.4 Dynamic free-surface condition

Neglecting viscosity and surface tension, the pressure at the free surface is atmospheric and therefore constant. Bernoulli's equation gives

$$\frac{1}{2}(u^2 + v^2) + gy + \frac{p}{\rho} + \frac{\partial \phi}{\partial t} = B(t),$$

where $B(t)$ is an arbitrary function of time. On the free surface, $p = p_{atm}$ is constant. In the wave frame, the steady form becomes

$$\frac{1}{2}[(u - c)^2 + v^2] + gy = R,$$

where R is a constant often interpreted as a Bernoulli constant or hydraulic head in the moving frame.

This condition is strongly nonlinear because it contains the square of the velocity magnitude evaluated on the unknown boundary $y = \eta(x)$.

4.5 Wave height constraint

The specified wave height is

$$H = \eta_{crest} - \eta_{trough}.$$

This becomes one of the global constraints used to close the algebraic system in stream-function/Fourier implementations.

4.6 Periodicity condition

The wave is periodic with wavelength L :

$$\eta(x + L) = \eta(x),$$

and likewise for all flow quantities in the wave frame.

5. Travelling-frame formulation and steady-wave unknowns

The classical fully nonlinear problem is converted into a time-independent problem by introducing the travelling coordinate

$$\xi = x - ct.$$

In the moving frame, the unknowns are no longer functions of both x and t , but of a single phase variable ξ .

The main unknowns in a steady-wave formulation are then:

- the free-surface profile $\eta(\xi)$,
- the stream function or velocity potential in the fluid domain,
- the wave speed c or wavelength L when the period is prescribed,
- the Bernoulli constant,
- the Fourier or stream-function coefficients describing the field.

If T is given, then c and L are related by

$$c = \frac{L}{T}.$$

In a current-inclusive setting, care is required because several velocity measures coexist:

- **phase speed** c ,
- **Eulerian current** U_c ,
- **relative speed in the wave frame**,
- **mass transport velocity** or **Stokes drift** diagnostics.

The codebase explicitly manages this distinction, especially in `fenton_gui.py`, `function.py`, and `fourier.cpp`.

6. Stream function formulation

The modern fully nonlinear steady-wave solver in this suite is best understood through the stream function.

6.1 Definition of the stream function

For a two-dimensional incompressible flow, define the stream function ψ by

$$u = \frac{\partial \psi}{\partial y}, \quad v = -\frac{\partial \psi}{\partial x}.$$

This automatically satisfies continuity. Irrotationality then implies that ψ is harmonic:

$$\nabla^2 \psi = 0.$$

6.2 Bed and free surface as streamlines

Because the bed is impermeable, it is a streamline. In the moving frame, the free surface is also a streamline by the kinematic boundary condition. Thus one may choose constants such that
\$\$

$$\begin{aligned} \psi &= -Q \quad \text{on } y = -d, \\ \psi &= 0 \quad \text{on } y = \eta(x), \end{aligned}$$

\$\$

or an equivalent pair depending on normalization. Here Q is related to the volume flux in the moving frame.

This is one of the key simplifications of the stream-function method: the kinematic condition is built into the representation through the choice of streamlines.

6.3 Harmonic basis satisfying the bed condition

A Fourier-like basis is selected to satisfy Laplace's equation and the bed boundary condition exactly. The free surface is then determined by enforcing the surface streamline condition and Bernoulli condition at discrete collocation points. A standard representation is of the form

$$\psi(x, y) = By + \sum_{n=1}^N b_n \frac{\sinh[nk(y + d)]}{\cosh(nkd)} \cos(nkx),$$

or a closely related equivalent normalization.

The exact coefficient names and scaling can differ between texts and codes, but the operational structure is the same:

- the vertical basis uses hyperbolic functions so that Laplace's equation is satisfied exactly,

- the horizontal dependence uses trigonometric modes to enforce periodicity,
- the bed condition is satisfied identically,
- the free-surface conditions are imposed numerically.

The associated velocities become

\$\$

$$u(x, y) = \frac{\partial \psi}{\partial y} = B + \sum_{n=1}^N b_n \frac{nk \cosh[nk(y + d)]}{\cosh(nkd)} \cos(nkx),$$

$$v(x, y) = -\frac{\partial \psi}{\partial x} = \sum_{n=1}^N b_n \frac{nk \sinh[nk(y + d)]}{\cosh(nkd)} \sin(nkx).$$

\$\$

These formulas are the backbone of the free-surface evaluation and of the velocity diagnostics produced in the implementation.

7. Fourier representation of the free surface

The free surface is periodic and symmetric about the crest for regular waves. It is therefore natural to write it as a cosine series:

$$\eta(x) = \sum_{n=0}^N a_n \cos(nkx),$$

or equivalently in a shifted normalization with a_0 interpreted as the mean level correction.

The coefficients a_n describe the crest sharpening, trough flattening, and higher-harmonic distortion absent in linear theory. In practice:

- a_1 is the dominant fundamental mode,
- a_2 captures first nonlinear asymmetry,
- higher modes encode increasingly sharp crest structure.

In the implementation, the surface unknowns may be represented either directly as coefficient vectors or as discrete free-surface ordinates at collocation points, depending on where one is in the solve/report pipeline.

8. Collocation and nonlinear algebraic system

The free surface is unknown, so the Bernoulli condition and streamline condition must be imposed on an unknown boundary. The standard numerical device is **collocation**.

8.1 Choice of collocation points

Take $N + 1$ surface points over half a wavelength (or N interior points, depending on the symmetry convention), commonly

$$x_m = \frac{m\pi}{Nk}, \quad m = 0, 1, \dots, N$$

or a related crest-to-trough distribution.

At each point (x_m, η_m) , enforce the nonlinear free-surface conditions.

8.2 Algebraic unknown vector

A typical unknown vector contains:

- Fourier coefficients for the stream function,
- Fourier coefficients or ordinates for the surface,
- wavenumber k or wavelength L ,
- Bernoulli constant R ,
- mean level correction,
- flux/current-related constants,
- one or more normalization parameters.

The exact dimension depends on the implementation and whether period, celerity, flux, or current is treated as prescribed.

8.3 Surface streamline equations

The condition that the free surface is a streamline implies

$$\psi(x_m, \eta_m) = 0$$

for each collocation point, up to the chosen normalization.

8.4 Dynamic equations at collocation points

At each collocation point,

$$\frac{1}{2} [(u - c)^2 + v^2] + g\eta_m = R.$$

Since both u and v depend nonlinearly on the unknown coefficients and are evaluated at the unknown ordinate η_m , this is a strongly nonlinear system.

8.5 Global closure equations

Additional scalar conditions are required, such as:

- the specified wave height,
- the specified period or celerity relation,
- a mean water-level condition,
- flux/current consistency,
- a crest or trough phase normalization.

This closes the system.

9. Relation to the Dean-Rienecker-Fenton method

The implementation philosophy used in `fourier.cpp` and echoed in the other solver paths belongs to the family introduced by Rienecker and Fenton. The method is often described as a **Fourier approximation method** or **stream-function method**. Its key ideas are:

1. Expand the unknown harmonic field in a basis satisfying the bed condition and Laplace equation exactly.
2. Represent the free surface by a truncated Fourier series or collocated ordinates.
3. Enforce the streamline and Bernoulli conditions at surface collocation points.
4. Solve the resulting nonlinear algebraic system by Newton iteration.
5. Use continuation in wave height, current, or depth to reach strongly nonlinear states robustly.

This framework is more accurate and more general than low-order Stokes expansions because it is not based on a small-amplitude truncation of the governing equations. Instead, it solves the full nonlinear steady problem within spectral truncation error.

10. Nonlinear solution by Newton iteration

The resulting system can be written compactly as

$$F(z) = 0,$$

where z is the vector of all unknowns. Newton's method updates

\$\$

$$\begin{aligned} J(z^{(m)}) \Delta z^{(m)} &= -F(z^{(m)}), \\ z^{(m+1)} &= z^{(m)} + \Delta z^{(m)}, \end{aligned}$$

\$\$

with Jacobian matrix

$$J_{ij} = \frac{\partial F_i}{\partial z_j}.$$

10.1 Why the Jacobian is difficult

The Jacobian contains derivatives of hyperbolic/trigonometric series evaluated on the unknown free surface. Terms arise from:

- direct coefficient differentiation,
- surface-elevation dependence in basis evaluation,
- Bernoulli quadratic velocity terms,
- wavelength dependence through k .

Strong nonlinearity near steep waves can make the Jacobian ill-conditioned.

10.2 Continuation strategy

To improve robustness, the solver often approaches the final target through a sequence of gradually increasing heights or currents. If the target state is (H, T, d, U_c) , one may solve first for a small-amplitude state, then ramp up one or more control parameters. This strategy is explicitly present in the logic of the nonlinear solvers and their initialization paths.

11. SVD and linear solve stabilization

In practice, the Newton correction system may be solved by LU, QR, or SVD-based methods. The implementations in this suite use stabilization strategies that include singular value decomposition in the VBA solver lineage and condition-aware linear algebra in the higher-level Python/C++ stack.

When the Newton step is written as

$$\mathbf{J}\Delta\mathbf{z} = -\mathbf{F},$$

an SVD factorization

$$\mathbf{J} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$$

allows one to compute a pseudoinverse-based correction,

$$\Delta\mathbf{z} = -\mathbf{V}\mathbf{\Sigma}^{-1}\mathbf{U}^T\mathbf{F},$$

with small singular values regularized or truncated if necessary. This materially improves robustness when the system is close to singularity, such as near limiting steepness or under poor initial guesses.

12. Integral constraints and hydraulic quantities

The steady-wave problem is not defined only by surface shape. It is also characterized by integral quantities that appear in Fenton’s papers and in the code output.

12.1 Volume flux

The mean flux in the wave frame is related to the stream function difference between the bed and the free surface. In many normalizations,

$$Q = \psi_{surface} - \psi_{bed}.$$

The exact sign depends on convention. This quantity is central in distinguishing solutions with the same geometric wave shape but different current definitions.

12.2 Mean water level and set-down / set-up

The mean free-surface elevation over a wavelength may be constrained or diagnosed as

$$\bar{\eta} = \frac{1}{L} \int_0^L \eta(x) dx.$$

Nonlinear periodic waves may exhibit a mean level shift relative to a chosen datum. Codes differ in whether they impose zero mean elevation, zero bed-to-mean depth change, or another normalization.

12.3 Energy density

The total wave energy per unit horizontal area is the sum of potential and kinetic components. A standard decomposition is

$$E = E_p + E_k,$$

with

$$E_p = \frac{\rho g}{L} \int_0^L \frac{\eta^2(x)}{2} dx,$$

and

$$E_k = \frac{\rho}{L} \int_0^L \int_{-d}^{\eta(x)} \frac{u^2 + v^2}{2} dy dx$$

relative to an appropriate reference state.

12.4 Momentum flux and radiation stress analogues

Momentum flux and related invariants can be derived from the pressure and velocity field. These quantities are relevant in hydrodynamic load and wave-current interaction analyses and appear conceptually across Fenton's theoretical work.

12.5 Mass transport and Stokes drift diagnostics

Nonlinear waves generally induce net mass transport. Distinguishing Eulerian current, Lagrangian drift, and wave frame flux is crucial. The solver and surrounding utilities are built to keep this distinction explicit, especially when the user specifies a current U_c .

13. Linear theory as baseline and initializer

Even though the suite is devoted to nonlinear waves, linear theory remains operationally important.

13.1 Classical Airy dispersion relation

Without current,

$$\omega^2 = gk \tanh(kd).$$

This can be written in wavelength form as

$$\left(\frac{2\pi}{T}\right)^2 = g \frac{2\pi}{L} \tanh\left(\frac{2\pi d}{L}\right).$$

13.2 Deep- and shallow-water limits

Deep water, $kd \gg 1$:

$$L_0 \approx \frac{gT^2}{2\pi}.$$

Shallow water, $kd \ll 1$:

$$c \approx \sqrt{gd}, \quad L \approx T\sqrt{gd}.$$

These asymptotic limits are used for initialization and for sanity checks throughout the toolchain.

13.3 Linear theory with current

If a uniform current U_c is present and the intrinsic frequency is Doppler shifted, then a common working relation is

$$\omega - kU_c = \sqrt{gk \tanh(kd)}.$$

Equivalently,

$$\left(\frac{2\pi}{T} - kU_c\right)^2 = gk \tanh(kd).$$

This relation is nonlinear in k and is solved iteratively in the linear baseline modules such as `linear.bas` and in the initialization logic of the surrogate builders. It is not the full nonlinear solution, but it provides a physically meaningful baseline wavelength L_{lin} .

14. Nonlinear corrections beyond Airy theory

The full Fenton solver does not merely add a small correction to the linear wavelength; it solves the entire steady boundary-value problem. However, the surrogate models in the suite are often built around the idea of a multiplicative or additive correction to a linear baseline.

A generic structure is

$$L = L_{lin} \mathcal{C}(H, T, d, U_c),$$

where $\mathcal{C}$ is a nonlinear correction factor learned from exact solver outputs.

Typical nondimensional variables used in these surrogates include:

- relative height: H/d ,
- relative depth: d/L_{lin} or kd ,
- wave steepness: H/L_{lin} ,
- dimensionless period: $T\sqrt{g/d}$,
- current Froude number: $U_c/\sqrt{gd}$,
- Doppler factor: $U_c T/L_{lin}$,
- Ursell number: $Ur = \frac{HL^2}{d^3}$, usually evaluated with a baseline or nonlinear wavelength.

These features appear explicitly in `pade.py`, `neural.py`, and `genetic.py`.

15. Why Fenton theory is preferred over simple Stokes formulas

Low-order Stokes expansions approximate the wave field as a perturbation series in a small steepness parameter. They are useful but limited.

15.1 Limitations of low-order perturbation theory

Stokes theories:

- lose accuracy at finite or large steepness,
- become awkward in intermediate and shallow depth,
- require order-by-order derivation,
- are not equally robust in the presence of current,
- may not preserve all nonlinear balances satisfactorily for engineering use.

15.2 Advantages of the Fourier/stream-function method

The Fenton-style method:

- solves the full nonlinear steady free-boundary problem,
- works across shallow, intermediate, and deep depths,
- naturally resolves crest sharpening and higher harmonics,
- supports current/flux formulations,
- provides the full velocity and surface field, not just a low-order approximation.

For this reason the suite uses the Fenton solver as the physical reference and only then derives surrogates for speed.

16. Theory of the Fourier approximation used in `fourier.cpp`

This is the central theoretical section because `fourier.cpp` is the most explicit low-level implementation of the solver architecture. The file is effectively a monolithic implementation of the

J.D. Fenton steady-wave Fourier / stream-function method and preserves the legacy unknown structure, nonlinear closures, and output logic in a particularly transparent way.

16.1 Structural idea of the code

The solver constructs a nonlinear algebraic system whose unknowns represent the free surface and harmonic field. It then applies a continuation / Newton iteration loop. Once converged, it evaluates the solved field to produce:

- summary solution quantities,
- surface coordinates,
- subsurface kinematics,
- Bernoulli diagnostics,
- integral quantities such as current, discharge-related quantities, and wave invariants.

The code embodies the following conceptual decomposition:

1. **Input normalization and dimensional / nondimensional conversion.**
2. **Initial guess construction**, often starting from a linear or weakly nonlinear state.
3. **Assembly of residual equations** for the collocation problem.
4. **Assembly or update of Jacobian information.**
5. **Linear solve for the Newton correction.**
6. **Continuation step control and convergence tests.**
7. **Post-processing of integral and field quantities.**
8. **Writing solution files.**

16.2 Physical, travelling, and solver coordinates

The code uses three related coordinate systems.

Laboratory coordinates

$$(x, y),$$

with:

- x positive in the direction of propagation,
- y positive upward,
- bed at $y = 0$ in the finite-depth implementation,
- mean free surface at $y = d$.

Travelling / mean-level coordinates

$$\xi = x - ct, \quad y' = y - d.$$

Thus:

- mean free surface is at $y' = 0$,
- bed is at $y' = -d$.

Non-dimensional solver coordinates The nonlinear system is written in

$$X = k\xi, \quad Y = ky', \quad k = \frac{2\pi}{\lambda}.$$

Hence one wavelength corresponds to

$$X \in [0, 2\pi],$$

and for finite depth the bed is located at

$$Y = -kd.$$

This coordinate choice is the reason the code repeatedly uses the quantity kd as a principal global unknown in the finite-depth problem.

16.3 Travelling-frame formulation and velocity definitions

In the travelling frame the wave is steady. If the laboratory-frame velocities are (u, v) , then the travelling-frame velocities are

$$U = u - c, \quad V = v.$$

In the stream-function formulation used by the code, the nondimensional travelling-frame velocities satisfy

$$\hat{U} = \frac{\partial \psi}{\partial Y}, \quad \hat{V} = -\frac{\partial \psi}{\partial X},$$

which is the standard Fenton stream-function formulation. Because the flow is incompressible and irrotational, the stream function is harmonic:

$$\psi_{XX} + \psi_{YY} = 0.$$

A velocity potential ϕ also exists, with

$$\hat{U} = \phi_X, \quad \hat{V} = \phi_Y,$$

and the Cauchy-Riemann relations become

$$\phi_X = \psi_Y, \quad \phi_Y = -\psi_X.$$

16.4 Boundary conditions used in the nonlinear system

Let the free surface be written as

$$Y = \eta(X).$$

Then the finite-depth boundary conditions are:

Bed impermeability

$$\psi(X, -kd) = 0.$$

Free-surface streamline condition

$$\psi(X, \eta(X)) = -q.$$

Free-surface Bernoulli condition

$$\frac{1}{2} (\psi_X^2 + \psi_Y^2) + \eta(X) = R.$$

Here:

- q is the streamline constant associated with the discharge in the travelling frame,
- R is the Bernoulli constant in the solver nondimensionalisation.

For deep water, the bed condition is replaced by exponential decay as

$$Y \rightarrow -\infty.$$

16.5 Fourier / stream-function representation actually used

The finite-depth stream-function representation can be written in a standard bed-based form as

$$\psi(\xi, y) = -\bar{U} y + \sum_{j=1}^N B_j \frac{\sinh(jky)}{\cosh(jkd)} \cos(jk\xi),$$

where:

- $k = 2\pi/\lambda$ is the wavenumber,
- $\bar{U}$ is the mean travelling-frame speed,
- B_j are Fourier coefficients,
- N is the truncation order.

Because the implementation works in the shifted vertical coordinate

$$Y = k(y - d),$$

the hyperbolic basis is rewritten using

$$\frac{\sinh(jky)}{\cosh(jkd)} = \frac{\sinh(j(Y + kd))}{\cosh(jkd)} = \sinh(jY) + \cosh(jY) \tanh(jkd).$$

Define the finite-depth basis function

$$S_j(Y) = \sinh(jY) + \cosh(jY) \tanh(jkd).$$

Then the solver evaluates the harmonic structure in terms of $S_j(Y)$. In the deep-water limit,

$$\tanh(jkd) \rightarrow 1,$$

so that

$$S_j(Y) \rightarrow e^{jY},$$

which is exactly the exponential vertical basis used in the deep-water branch.

16.6 Surface and harmonic unknowns

The nonlinear solution uses a truncated spectral representation. Let the stream-function basis coefficients be B_j and the surface representation coefficients or ordinates be a_n or η_m . The solver keeps these coefficients in fixed-size arrays and updates them during Newton steps.

The free surface can be represented by a cosine series of the form

$$\eta(X) = \sum_{n=0}^N a_n \cos(nX),$$

or equivalently through collocated nodal ordinates

$$\eta_m = \eta(X_m).$$

The code uses a legacy 1-based unknown vector z that can be interpreted schematically as

$$z = [\text{global scalars} \mid \text{surface ordinates} \mid \text{Fourier coefficients}].$$

A convenient interpretation of the leading scalar unknowns is:

\$\$

$$\begin{aligned} z_1 &\leftrightarrow kd, & z_2 &\leftrightarrow kH, & z_3 &\leftrightarrow \frac{kc}{\omega}, \\ z_4 &\leftrightarrow c\sqrt{\frac{k}{g}}, & z_5 &\leftrightarrow \bar{u}_1\sqrt{\frac{k}{g}}, & z_6 &\leftrightarrow \bar{u}_2\sqrt{\frac{k}{g}}, \\ z_7 &\leftrightarrow \bar{U}\sqrt{\frac{k}{g}}, & z_8 &\leftrightarrow \bar{U}kd - q, & z_9 &\leftrightarrow R. \end{aligned}$$

\$\$

Then:

- z_{10+m} , with $m = 0, \dots, N$, stores the collocated values of $k\eta_m$,
- the remaining entries store the Fourier coefficients B_j .

This interpretation is important because it explains how the code couples wavelength, current, free-surface elevation, and harmonic coefficients in a single nonlinear solve.

16.7 Current definitions and closure relations

The code distinguishes between two mean-current definitions:

- Eulerian mean current $\bar{u}_1$,
- Stokes / mass-transport current $\bar{u}_2$.

The travelling-frame mean speed $\bar{U}$ is related to the Eulerian current by

$$\bar{u}_1 = c - \bar{U}.$$

The discharge-related current is related to the mean discharge Q by

$$\bar{u}_2 = c - \frac{Q}{d}.$$

This distinction matters because a period-specified problem with current is not fully defined until one decides which mean-current measure is being prescribed.

16.8 Collocation grid and residual structure

The code exploits crest-trough symmetry and solves only over half a wavelength, with collocation points

$$X_m = \frac{m\pi}{N}, \quad m = 0, 1, \dots, N.$$

At each collocation point the residual vector contains two principal classes of equations.

Streamline residual

$$R_m^{(\psi)} = \psi(X_m, \eta_m) + q = 0.$$

In the internal shifted-coordinate implementation this is assembled using the equivalent constant combination involving $\bar{U}kd - q$, but the underlying hydrodynamic meaning is exactly the streamline condition above.

Bernoulli residual

$$R_m^{(B)} = \frac{1}{2} [(u_m - c)^2 + v_m^2] + \eta_m - R = 0$$

in solver nondimensional form. Because the code explicitly carries the travelling-frame mean speed, the relative horizontal velocity is represented through

$$u_m - c = \tilde{u}_m - \bar{U},$$

where $\tilde{u}_m$ is the wave-induced contribution evaluated from the Fourier basis.

16.9 Height, phase, period, and current closure equations

The nonlinear system is closed by additional global constraints. These include:

Wave-height closure

$$H = \eta_{\text{crest}} - \eta_{\text{trough}},$$

or, in k -scaled form,

$$kH = k\eta_{\text{crest}} - k\eta_{\text{trough}}.$$

Wavelength-specified closure If wavelength is specified, the closure is equivalent to prescribing

$$\frac{\lambda}{d},$$

so that

$$kd = \frac{2\pi d}{\lambda}.$$

Period-specified closure If period is specified, then

$$c = \frac{\lambda}{T}, \quad L = cT,$$

but now L is part of the solved state rather than an imposed Airy value.

Current closure One additional scalar equation connects the selected current criterion to the unknowns $(c, \bar{u}_1, \bar{u}_2, \bar{U}, q)$.

16.10 Jacobian structure

Differentiating the residual equations produces Jacobian contributions such as

$$\frac{\partial R_m^{(\psi)}}{\partial B_n}, \quad \frac{\partial R_m^{(\psi)}}{\partial \eta_j}, \quad \frac{\partial R_m^{(\psi)}}{\partial k},$$

and

$$\frac{\partial R_m^{(B)}}{\partial B_n}, \quad \frac{\partial R_m^{(B)}}{\partial \eta_j}, \quad \frac{\partial R_m^{(B)}}{\partial R}, \quad \frac{\partial R_m^{(B)}}{\partial c}.$$

Because

$$R_m^{(B)} = \frac{1}{2} [(u_m - c)^2 + v_m^2] + g\eta_m - R,$$

one obtains terms like

$$\frac{\partial R_m^{(B)}}{\partial z_j} = (u_m - c) \frac{\partial u_m}{\partial z_j} + v_m \frac{\partial v_m}{\partial z_j} + g \frac{\partial \eta_m}{\partial z_j} - \frac{\partial R}{\partial z_j}.$$

This dense coupling is one reason the algebra becomes stiff and ill-conditioned near steep or long waves.

16.11 Newton iteration and SVD stabilization

The nonlinear system is solved by Newton iteration:

\$\$

$$\begin{aligned} F(\mathbf{z}) &= 0, \\ \mathbf{J}(\mathbf{z}^{(m)}) \Delta \mathbf{z}^{(m)} &= -F(\mathbf{z}^{(m)}), \\ \mathbf{z}^{(m+1)} &= \mathbf{z}^{(m)} + \Delta \mathbf{z}^{(m)}. \end{aligned}$$

\$\$

The code uses an SVD-based dense solve to stabilize the Newton correction. In abstract form,

$$\mathbf{J} = \mathbf{U} \Sigma \mathbf{V}^T,$$

so that a pseudoinverse-like step may be written as

$$\Delta \mathbf{z} = -\mathbf{V} \Sigma^{-1} \mathbf{U}^T \mathbf{F},$$

with small singular values truncated or regularized. This is especially important near limiting waves, long shallow-water states, or strongly adverse currents.

16.12 Continuation and height stepping

The code can solve a sequence of increasing wave heights and extrapolate the unknowns between steps. This continuation strategy is one of the classic practical devices in Fenton-style solvers because the final target state can be difficult to reach directly for steep waves. In effect, the solver traces a branch of solutions rather than attempting to jump immediately to the final nonlinear state.

16.13 Mean square bed orbital velocity

The program reports a quantity labelled mean square bed velocity, denoted `ub2`. In physical terms it is the phase average of the near-bed **orbital** horizontal velocity relative to the Eulerian mean current:

$$u_{b2} = \langle (u_{\text{bed}}(t) - \bar{u}_1)^2 \rangle.$$

For a steady travelling wave, averaging over one period is equivalent to phase-averaging over one wavelength.

16.14 Output reconstruction

After convergence, the program evaluates:

- $\eta(x)$ on a dense grid for `surface.res`,
- $u(x, y)$ and $v(x, y)$ on a field grid for `flowfield.res`,
- summary invariants and diagnostic scalars for `solution.res`.

The output files therefore derive from the exact same spectral representation used in the solve; they are not separate approximate post-processors.

16.15 Why `fourier.cpp` is central

`fourier.cpp` is the clearest low-level embodiment of Fenton’s full nonlinear hydrodynamic model, ported directly from Fenton’s c++ original source code. Even when operational workflows are driven by `fenton_gui.py`, `function.py`, or the surrogate tools, the mathematical authority still comes from this Fourier / stream-function framework. For that reason, this chapter anchors the theoretical interpretation of the whole suite.

17. Detailed theory embedded in `fenton_gui.py`

The Python GUI is the main operational front-end. Although it is a user-facing program, its internal logic is still mathematically structured.

17.1 Dual-case philosophy: without current and with current

The GUI computes two physically related but distinct problems:

1. a nonlinear wave solution without prescribed current,
2. a nonlinear wave solution with current U_c .

This dual presentation is important because it lets the user compare the effect of current on wavelength, celerity, and wave shape diagnostics. The report formatting logic is not superficial; it encodes a hydrodynamic comparison workflow.

17.2 Input variables

The main inputs are:

- wave height H ,
- period T ,
- water depth d ,
- current U_c ,
- solver order / number of harmonics,
- numerical tolerances and internal settings,

- report options.

17.3 The wavelength solve as the central computational target

Within the GUI architecture, many displayed quantities are downstream of the computed wavelength. Once L is known, other diagnostic nondimensional groups can be evaluated:

- steepness H/L ,
- relative depth d/L ,
- wave number $k = 2\pi/L$,
- celerity $c = L/T$,
- relative current indicators such as U_c/c and $U_c/\sqrt{gd}$.

17.4 Full-field vs engineering-report outputs

The GUI provides a user-friendly interface translating the nonlinear solution into engineering-readable outputs such as:

- wavelengths,
- celerities,
- regime indicators,
- possibly crest/trough elevations,
- comparison tables,
- textual interpretation.

Thus the code combines mathematical solving with presentation and validation logic.

17.5 Relationship to `function.py`

The GUI and auxiliary utilities rely on a callable external wavelength function. This modularization means the exact nonlinear wavelength logic can be reused in tables, surrogates, and external workflows without repeating the full GUI stack.

18. Standalone wavelength function: theoretical role of `function.py`

`function.py` is strategically important because it compresses the nonlinear wavelength calculation into a reusable API.

18.1 Abstract mathematical role

It implements a callable mapping

$$L = f(H, T, d, U_c).$$

This is exactly the engineering object the rest of the project needs. Unlike the full solver, which can return the entire wave field, `function.py` acts as a scalar nonlinear constitutive map from input wave parameters to wavelength.

18.2 Why this abstraction matters

Once a physically credible nonlinear wavelength function exists, many derivative tools become possible:

- batch tabulation,
- spreadsheet usage,
- surrogate model training,
- Monte Carlo sampling,
- design tables,
- parametric studies.

18.3 CLI use

The script is designed so it can be called directly, for example:

```
python function.py 3 9 5 1
```

which corresponds to

- $H = 3$ m,
- $T = 9$ s,
- $d = 5$ m,
- $U_c = 1$ m/s.

The returned value $L = 78.8272$ m is Fenton's fully nonlinear wavelength associated with those inputs.

19. Table generation with `tables.py`

`tables.py` is not just a formatting script. It is an automation layer over the nonlinear wavelength kernel.

19.1 Mathematical function

It samples the map

$$(H, T, d, U_c) \mapsto L$$

over a prescribed grid of input values and writes organized tables. These tables are useful because they provide a simple no-calculations way of obtaining a good estimate of Fenton's exact wavelength.

19.2 Engineering relevance

By tabulating the exact or reference nonlinear function, the project creates practical design aids. A user can inspect how wavelength varies with:

- increasing height at fixed period and depth,
- increasing depth at fixed height,
- following current or opposing current,

- period changes across depth regimes.

19.3 Derived nondimensional columns

A well-designed table generator may also compute:

$$\frac{H}{L}, \quad \frac{d}{L}, \quad kd, \quad \frac{U_c}{c}, \quad \frac{U_c}{\sqrt{gd}}.$$

These quantities provide immediate physical interpretation.

20. Padé surrogate theory in `pade.py`

The Padé module creates a rational approximation to the nonlinear wavelength correction.

20.1 Why Padé approximants are useful

A Padé approximant represents a function as a ratio of polynomials:

$$P(x) = \frac{a_0 + a_1x + a_2x^2 + \cdots + a_mx^m}{1 + b_1x + b_2x^2 + \cdots + b_nx^n}.$$

Compared with an ordinary polynomial, a rational form can often capture curvature, asymptotes, and cross-regime behavior more efficiently.

20.2 Regime splitting

The script often classifies data by relative depth, such as:

- shallow,
- intermediate,
- deep.

A representative criterion uses

$$\mu = \frac{d}{L},$$

with thresholds like

\$\$

$$\begin{aligned} \mu &< 0.05 && \text{(shallow),} \\ 0.05 \leq \mu &< 0.5 && \text{(intermediate),} \\ \mu &\geq 0.5 && \text{(deep).} \end{aligned}$$

\$\$

Separate Padé forms can then be fitted in each regime.

20.3 Correction-factor approach

A common operational strategy is

$$L = L_{base} \cdot C_{\text{Padé}}(\mathbf{x}),$$

where L_{base} may be linear or another baseline, and $\mathbf{x}$ is a feature vector built from nondimensional inputs.

20.4 Features used by the Padé model

The script logic uses physically meaningful engineered features such as:

\$\$

$$\begin{aligned}\text{WaveSteepness} &= \frac{H}{L_{lin}}, \\ \text{RelativeDepth} &= \frac{d}{L_{lin}}, \\ \text{DopplerFactor} &= \frac{U_c T}{L_{lin}}, \\ \text{UrsellNumber} &= \frac{H L_{lin}^2}{d^3}, \\ \text{CurrentFroude} &= \frac{U_c}{\sqrt{gd}}.\end{aligned}$$

\$\$

The feature-selection stage ranks these by correlation or predictive relevance and fits compact rational models accordingly.

20.5 Emitted artifacts

The script emits:

- a standalone Python evaluator,
- a VBA evaluator (`pade.bas`),
- diagnostics,
- fit reports.

Thus the theoretical role of `pade.py` is to compress the exact nonlinear solver into a portable analytic surrogate.

21. Neural surrogate theory in `neural.py`

The neural module provides a universal-function approximation approach to the nonlinear wavelength map.

21.1 Functional form

A single-hidden-layer feedforward network with tanh activation has the form

$$\hat{y}(\mathbf{x}) = b_o + \sum_{j=1}^M w_j^{(o)} \tanh \left(b_j + \sum_{i=1}^p w_{ji}^{(h)} x_i \right),$$

where:

- $\mathbf{x}$ is the input feature vector,
- M is the number of hidden neurons,
- $w_{ji}^{(h)}$ are input-to-hidden weights,
- b_j are hidden biases,
- $w_j^{(o)}$ are hidden-to-output weights,
- b_o is the output bias.

The output may represent either L directly or a correction to a baseline wavelength.

21.2 Normalization

Neural networks are usually trained with scaled inputs and outputs:

$$\tilde{x}_i = \frac{x_i - \mu_i}{\sigma_i},$$

or via min-max scaling

$$\tilde{x}_i = \frac{x_i - x_i^{min}}{x_i^{max} - x_i^{min}}.$$

The emitted evaluator must therefore reproduce the exact preprocessing constants used in training.

21.3 Loss function

Training minimizes a loss such as mean squared error

$$\mathcal{L} = \frac{1}{N} \sum_{n=1}^N (y_n - \hat{y}_n)^2,$$

or a relative-error variant if wavelength scaling varies significantly across the dataset.

21.4 Why neural surrogates are useful here

The mapping $(H, T, d, U_c) \mapsto L$ is smooth but highly nonlinear across the full domain. Neural networks can represent such maps compactly without manual algebraic feature design. However, they are less transparent than Padé or symbolic formulas, which is why the suite includes multiple surrogate families rather than relying on only one.

21.5 Emitted standalone evaluator

`neural.py` emits a self-contained `function.py`-style evaluator and a VBA module `neural.bas` so the trained model can be used without Python ML dependencies.

22. Genetic symbolic-regression theory in `genetic.py` and project-discovered explicit approximate formulas

The genetic module aims to discover an explicit formula rather than a black-box predictor. In this project it is especially important because it provides one of the two project-specific explicit wavelength surrogates discussed in this chapter, both anchored to physically meaningful wave-dispersion structure.

22.1 Symbolic regression objective

The goal is not to fit the wavelength with a purely arbitrary expression, but to evolve a symbolic correction to a baseline theory. The generic structure is

$$L \approx L_{\text{base}} C_{\text{GEP}}(\mathbf{x}),$$

or, more abstractly,

$$L \approx G(\mathbf{x}),$$

where G is assembled from a grammar of algebraic operators and nonlinear elementary functions acting on engineered nondimensional features.

22.2 Baseline-plus-correction philosophy

The genetic model is scientifically appealing because it preserves three desirable properties at once:

1. a **physical baseline** coming from linear wave theory,
2. a **compact explicit correction** learned from nonlinear data,
3. direct **deployability** to spreadsheet, VBA, and lightweight command-line environments.

In practical terms, the baseline is the linear wavelength L_{lin} , while the symbolic-regression model learns a multiplicative correction.

22.3 Linear baseline used by the genetic model

The baseline wavelength is obtained from the finite-depth Airy dispersion relation

$$\omega^2 = gk \tanh(kd), \quad \omega = \frac{2\pi}{T}.$$

Once k is obtained iteratively, the baseline wavelength is

$$L_{\text{lin}} = \frac{2\pi}{k}.$$

Equivalently, the baseline can be written as the implicit wavelength equation

$$L_{\text{lin}} = \frac{gT^2}{2\pi} \tanh\left(\frac{2\pi d}{L_{\text{lin}}}\right).$$

This means the symbolic-regression model does **not** attempt to rediscover dispersion from nothing. Instead, it learns the nonlinear and current-related correction relative to a hydrodynamically meaningful starting point.

22.4 Gene expression programming structure

Gene expression programming (GEP) evolves populations of encoded symbolic expressions. The algorithm typically includes:

- chromosome initialization,
- fitness evaluation,
- selection,
- mutation,
- recombination,
- elitism,
- repeated generation cycling.

The search therefore explores a large symbolic-expression space while still allowing the final result to be exported as an explicit closed-form formula.

22.5 Typical objective functions

A representative fitness metric is based on mean absolute percentage error:

$$\text{MAPE} = \frac{100}{N} \sum_{n=1}^N \left| \frac{y_n - \hat{y}_n}{y_n} \right|.$$

Other metrics may include RMSE, percentile error, worst-case error, or composite ranking scores. This is important because symbolic-regression formulas can appear good on average while failing in the tails of the parameter domain.

22.6 Why symbolic regression matters

For engineering workflows, an explicit formula has several advantages:

- transparency,
- direct embedding in reports and spreadsheets,
- no hidden weights,
- no iterative nonlinear solve at runtime beyond the baseline dispersion solve,
- easier auditing and independent checking.

The trade-off is that uniformly small error over a broad nonlinear domain is difficult to achieve with a compact explicit expression.

22.7 Project feature set used in the genetic workflow

The symbolic-regression workflow uses a set of engineered nondimensional features constructed from the physical variables and the baseline wavelength. A representative set is:

\$\$

$$\begin{aligned} x_0 &= \ln\left(\frac{d}{L}\right), & x_1 &= \ln\left(\frac{H}{L}\right), & x_2 &= \ln\left(\frac{H}{d}\right), \\ x_3 &= \ln(Ur), & Ur &= \frac{HL^2}{d^3}, \\ x_4 &= \frac{U_c}{\sqrt{gd}}, & x_5 &= \frac{U_c T}{L}, & x_6 &= \frac{U_c}{C_0}, \end{aligned}$$

\$\$

with

$$C_0 = \frac{gT}{2\pi}, \quad L_0 = \frac{gT^2}{2\pi},$$

and also

$$x_7 = \ln\left(\frac{H}{L_0}\right), \quad x_8 = \ln\left(T\sqrt{\frac{g}{d}}\right).$$

The exact active subset depends on the evolved symbolic expression, but these variables capture relative depth, steepness, Ursell nonlinearity, current Froude-like intensity, and Doppler-type effects.

22.8 Project-specific discovered approximate formulas

This project contains **two project-specific explicit approximate wavelength formulas** derived from the nonlinear-wave dataset and operationalized in the toolchain.

Formula A: genetic baseline-plus-correction formula A particularly important result obtained in this project is the following explicit genetic evolutionary wavelength formula:

$$L_{\text{genetic}} = L_{\text{lin}} \left[\exp\left(\frac{345 U_c T}{509 L_{\text{lin}}}\right) + \frac{1052\pi U_c}{1052\pi U_c + 435gT} - \frac{5 \tanh\left(\frac{U_c T}{L_{\text{lin}}} \ln\left(T\sqrt{\frac{g}{d}}\right) + \frac{2117}{961}\right)}{67 \ln\left(\frac{H}{d}\right)} \right].$$

Here:

- L_{genetic} is the wavelength predicted by the project-specific symbolic-regression model,
- L_{lin} is the baseline linear wavelength,
- H is wave height,
- T is wave period,
- d is water depth,
- U_c is current velocity,

- g is gravitational acceleration.

Formula B: effective transformed-variable formula A second project-specific explicit approximate formula used in this project is the effective transformed-variable formulation:

\$\$

$$T_{\text{eff}} = T - \frac{93}{943} \left(U_c^2 - \frac{H}{1819 \cdot 708} \right) + \frac{500}{739} U_c,$$

$$d_{\text{eff}} = d + \frac{H^{3/4}}{d^{1/4}} + \frac{986}{873} U_c \tanh(H).$$

\$\$

and the wavelength is then obtained from a linear finite-depth dispersion solve using these transformed variables:

$$L_{\text{effective}} = \mathcal{L}_{\text{lin}}(T_{\text{eff}}, d_{\text{eff}}),$$

where $\mathcal{L}_{\text{lin}}$ denotes the usual linear finite-depth wavelength operator defined implicitly by

$$\left(\frac{2\pi}{T_{\text{eff}}} \right)^2 = gk \tanh(kd_{\text{eff}}), \quad L_{\text{effective}} = \frac{2\pi}{k}.$$

22.9 Structural form of the two discovered formulas

The two discovered formulas have different mathematical architectures.

A. Genetic baseline-plus-correction form The genetic formula applies an explicit multiplicative correction to the linear baseline:

$$L_{\text{genetic}} = L_{\text{lin}} M,$$

with

\$\$

$$M = t_1 + t_2 + t_3,$$

$$t_1 = \exp\left(\frac{345 U_c T}{509 L_{\text{lin}}}\right), \quad t_2 = \frac{1052\pi U_c}{1052\pi U_c + 435gT},$$

$$t_3 = -\frac{5 \tanh\left(\frac{U_c T}{L_{\text{lin}}} \ln(T\sqrt{\frac{g}{d}}) + \frac{2117}{961}\right)}{67 \ln\left(\frac{H}{d}\right)}.$$

\$\$

B. Effective transformed-variable form The effective formula modifies the inputs to dispersion rather than the wavelength directly:

$$T_{\text{eff}} = T + \Delta_T, \quad d_{\text{eff}} = d + \Delta_d,$$

with

\$\$

$$\Delta_T = \left(U_c^2 - \frac{H}{1819 \cdot 708} \right) \left(-\frac{93}{943} \right) + \frac{500}{739} U_c,$$

$$\Delta_d = \sqrt{\frac{H}{\sqrt{d/H}}} + \frac{986}{873} U_c \tanh(H) = \frac{H^{3/4}}{d^{1/4}} + \frac{986}{873} U_c \tanh(H).$$

\$\$

The resulting wavelength is

$$L_{\text{effective}} = \mathcal{L}_{\text{lin}}(T_{\text{eff}}, d_{\text{eff}}).$$

In short, the genetic formula corrects wavelength directly, whereas the effective formula corrects the variables entering the linear dispersion relation.

22.10 Physical interpretation

The genetic formula is a direct nonlinear correction to L_{lin} . Its structure combines current, depth, and relative-height effects in one compact multiplier. The effective formula follows a different logic: it builds an apparent period and an apparent depth, then solves a standard linear wavelength problem using those transformed variables.

This distinction is the key conceptual separation between the two project formulas:

- **genetic.bas** = baseline wavelength times explicit correction,
- **effective.bas** = transformed inputs followed by linear dispersion.

22.11 Role of the main variables

In the genetic formula, the dominant quantities are the baseline wavelength L_{lin} , the Doppler-like group $U_c T / L_{\text{lin}}$, the relative-height term H/d , and the dimensionless period factor $T \sqrt{g/d}$.

In the effective formula, wave height and depth enter through

$$\sqrt{\frac{H}{\sqrt{d/H}}} = \frac{H^{3/4}}{d^{1/4}},$$

while current modifies both transformed variables through the terms proportional to U_c , U_c^2 , and $U_c \tanh(H)$.

22.12 Computational structure

Both discovered formulas are operationally simple.

For the genetic formula:

1. solve the standard linear dispersion relation for L_{lin} ,
2. evaluate the explicit correction factor M ,
3. compute $L_{\text{genetic}} = L_{\text{lin}}M$.

For the effective formula:

1. compute T_{eff} ,
2. compute d_{eff} ,
3. solve the linear finite-depth dispersion relation with those transformed variables.

Neither formula requires the full continuation / Newton / SVD machinery of the exact Fenton solver.

22.13 Domain restrictions and numerical cautions

The genetic formula contains logarithms and rational denominators. Therefore

$$\ln\left(\frac{H}{d}\right)$$

requires $H/d \neq 1$, and the denominator

$$1052\pi U_c + 435gT$$

must remain away from zero. The argument

$$T\sqrt{\frac{g}{d}}$$

must also remain positive.

The effective formula contains nested square roots, so its raw mathematical form requires

$$H > 0, \quad d > 0,$$

and the transformed period must remain non-zero:

$$T_{\text{eff}} \neq 0.$$

For spreadsheet robustness, **effective.bas** applies small numerical guards before solving the transformed linear dispersion relation.

22.14 Why these formulas matter

These formulas matter because they provide two distinct explicit surrogate philosophies derived within the project:

1. a direct baseline-correction formula (**genetic.bas**),
2. a transformed-variable linear-dispersion formula (**effective.bas**).

They are compact enough for VBA and spreadsheet deployment while remaining tied to physically meaningful wavelength structure.

22.15 Relation to operational VBA modules

The workbook implements both formulas directly:

- `genetic.bas` evaluates the symbolic-regression correction over L_{lin} ,
- `effective.bas` evaluates T_{eff} and d_{eff} , then solves the linear dispersion relation.

This makes both formulas immediately usable in engineering spreadsheet workflows.

22.16 Recommended interpretation in the project

The two discovered formulas should be treated as fast explicit surrogates, not as replacements for the fully nonlinear Fourier / stream-function solver.

- Use `genetic.bas` when a direct explicit correction to the baseline wavelength is preferred.
- Use `effective.bas` when a transformed-variable linear-dispersion structure is preferred.
- Use the full Fenton solver when the nonlinear wave field itself, not only wavelength, is required.

23. Nomogram theory in `nomogram_plots.py` and `nomogen_plots.py`

Nomograms provide a graphical computational device that approximates the function $L(H, T, d, U_c)$ or a reduced feature mapping derived from it.

23.1 General principle

A nomogram arranges one or more scales such that a straightedge alignment or chart lookup approximates a multivariable functional relation. If the surrogate is simple enough,

$$L = F(H, T, d, U_c),$$

or in reduced feature form

$$L = L_{\text{base}}C(x),$$

then the variables can be represented graphically.

23.2 Why nomograms still matter

They are useful when:

- no computation environment is available,
- quick field checks are needed,
- documentation benefits from visual design aids,
- one wants a compact engineering overview of parameter sensitivity.

23.3 Role of the two scripts

- `nomogram_plots.py` builds the plotting and report side directly.
- `nomogen_plots.py` integrates with a nomogram-generation engine and layout logic.

Both depend on a compact surrogate rather than the full Newton-based solver.

24. Excel/VBA implementations

The workbook layer is operationally important because many engineering workflows still rely on Excel.

24.1 `linear.bas`

This module solves the linear Doppler-shifted dispersion relation. In terms of wavenumber,

$$F(k) = \left(\frac{2\pi}{T} - kU_c \right)^2 - gk \tanh(kd) = 0.$$

A Newton iteration updates

$$k_{n+1} = k_n - \frac{F(k_n)}{F'(k_n)}.$$

The derivative is

$$F'(k) = -2U_c \left(\frac{2\pi}{T} - kU_c \right) - g \tanh(kd) - gkd \frac{1}{\cosh^2(kd)}.$$

Then

$$L = \frac{2\pi}{k}.$$

This module is the baseline initializer and sanity-check engine.

24.2 `fenton.bas`

This is the VBA translation of Fenton's nonlinear Fourier/stream-function solver. It uses:

- continuation,
- Newton iteration,
- Jacobian assembly,
- SVD stabilization,
- spectral post-processing.

Thus this VBA script it is the spreadsheet exact equivalent of the reference nonlinear solver.

24.3 `pade.bas`

This module evaluates the emitted rational approximations. A representative form is

$$L = L_{base} \frac{p_0 + p_1x_1 + p_2x_2 + \dots}{1 + q_1x_1 + q_2x_2 + \dots},$$

with regime-dependent coefficient sets.

24.4 `neural.bas`

This module implements the trained network in pure VBA using stored weights and biases, reproducing the preprocessing and the tanh activation exactly.

24.5 `genetic.bas`

This module evaluates the explicit symbolic-regression formula found by GEP. It is the most transparent surrogate in the Excel layer.

24.6 `wavelength.xlsm`

The workbook is the front-end that exposes those modules to the user. Its main roles are:

- input collection,
 - function selection,
 - result display,
 - spreadsheet integration.
-

25. Dimensional analysis and nondimensional groups used across the suite

The codebase repeatedly uses nondimensional variables because they improve both physical interpretation and numerical stability.

25.1 Depth-based scaling

Using depth d as scale:

$$\hat{H} = \frac{H}{d}, \quad \hat{L} = \frac{L}{d}, \quad \hat{c} = \frac{c}{\sqrt{gd}}, \quad \hat{U}_c = \frac{U_c}{\sqrt{gd}}.$$

25.2 Wavelength-based scaling

Using wavelength:

$$\frac{H}{L}, \quad \frac{d}{L}, \quad kd = \frac{2\pi d}{L}.$$

25.3 Period-based scaling

A common dimensionless period is

$$\tau = T \sqrt{\frac{g}{d}}.$$

Then linear theory can be written in terms of

$$\sigma = \frac{2\pi}{\tau}, \quad \alpha = \sigma^2.$$

These variables appear in approximate dispersion initializers and surrogates.

25.4 Ursell number

The Ursell number is

$$Ur = \frac{HL^2}{d^3}.$$

It measures the relative importance of nonlinearity and dispersion and is especially useful in shallow/intermediate depth.

25.5 Current Froude number

$$Fr_c = \frac{U_c}{\sqrt{gd}}.$$

This quantifies the importance of current relative to shallow-water gravity-wave speed.

25.6 Doppler factor

$$D = \frac{U_c T}{L}.$$

This is useful as a feature describing the magnitude of current-induced phase advection over one period.

26. Mathematical interpretation of current effects

Current affects the problem in two distinct but coupled ways.

26.1 Kinematic Doppler effect

A following current increases observed wavelength for fixed period, while an opposing current decreases it. In linearized form this follows from

$$\omega - kU_c = \omega_{intrinsic}(k).$$

26.2 Nonlinear dynamic modification

In the full nonlinear theory, current also changes the steady free-boundary structure through the relative flow field entering Bernoulli's equation. Thus the effect is not merely a shifted linear wavelength; it is a modified nonlinear state.

26.3 Blocking and invalid states

A sufficiently adverse current can lead to a failure of physically admissible propagating solutions. In reduced-order tools this may appear as a rejected input where Doppler cancellation makes the wavelength undefined or nonphysical.

27. Numerical conditioning and solver limits

A fully nonlinear wave solver is numerically delicate near extreme conditions.

27.1 High steepness

As the wave approaches the limiting steepness, crest curvature sharpens and higher harmonics grow. More modes and stronger continuation control are needed.

27.2 Very shallow water

Hyperbolic/trigonometric basis terms and current coupling can produce strongly scaled coefficients, increasing conditioning sensitivity.

27.3 Opposing current

Opposing current can reduce intrinsic phase speed and produce near-blocking states, which are difficult for both exact and surrogate models.

27.4 Why multiple surrogate families exist

No single reduced-order model is optimal everywhere. Rational, neural, and symbolic surrogates each trade off transparency, compactness, and accuracy differently.

28. Full usage manual

This section is intentionally operational after the theory sections above.

28.1 `fenton_gui.py`

Run:

```
python fenton_gui.py
```

Workflow:

1. Enter wave height H .
2. Enter period T .
3. Enter depth d .

4. Enter current U_c if requested.
5. Choose solver/report options.
6. Run the nonlinear solve.
7. Review the no-current and with-current results.
8. Export or inspect the report.

28.2 function.py

Direct call:

```
python function.py H T d Uc
```

Example:

```
python function.py 3 9 5 1
```

Programmatic use:

```
from function import fenton_wavelength
L = fenton_wavelength(3.0, 9.0, 5.0, 1.0)
```

28.3 tables.py

Run:

```
python tables.py
```

Expected behavior:

- samples parameter grids,
- calls the external wavelength function,
- writes formatted tables.

28.4 pade.py

Run:

```
python pade.py
```

Expected outputs:

- fitted surrogate coefficients,
- diagnostics,
- standalone evaluator,
- `pade.bas`.

28.5 neural.py

Run:

```
python neural.py
```

Expected outputs:

- trained network,
- diagnostic reports,
- standalone evaluator,
- `neural.bas`.

28.6 genetic.py

Run:

```
python genetic.py
```

Expected outputs:

- evolved symbolic formula,
- fit diagnostics,
- `genetic.bas`.

28.7 nomogram_plots.py

Run:

```
python nomogram_plots.py
```

Expected outputs:

- printed or saved nomogram figures,
- compact report graphics.

28.8 nomogen_plots.py

Run:

```
python nomogen_plots.py
```

Expected outputs:

- nomogram layouts using the alternate plotting engine.

28.9 fourier.cpp

Compile, for example:

```
g++ fenton_gui.cpp -o fenton_gui.exe -O3 -std=c++20 -march=native -lgdi32 -luser32  
↪ -lkernel32 -lcomctl32 -static-libgcc -static-libstdc++ -mwindows -pthread
```

Run:

```
fourier.exe
```

Typical behavior:

- reads inputs,
- solves nonlinear steady wave,
- writes summary solution and field files.

28.10 Excel workbook

Open `wavelength.xlsm`, enable macros, and use the workbook interface to call the desired VBA module.

29. Interpretation of outputs

29.1 Wavelength L

This is the principal nonlinear output. It should not be confused with the linear Airy wavelength unless the wave is weakly nonlinear.

29.2 Celerity c

$$c = \frac{L}{T}.$$

29.3 Wave number k

$$k = \frac{2\pi}{L}.$$

29.4 Relative depth

$$\frac{d}{L}$$

indicates whether the state is shallow, intermediate, or deep.

29.5 Steepness

$$\frac{H}{L}$$

indicates wave nonlinearity and crest sharpness tendency.

29.6 Current influence

Comparing with-current and no-current outputs reveals the Doppler and nonlinear current effects.

30. Conceptual map of the whole project

A concise way to understand the suite is:

1. **Physics reference:** fully nonlinear stream-function/Fourier solver.
2. **Reusable API:** standalone nonlinear wavelength function.
3. **Acceleration layer:** Padé, neural, and genetic surrogates.
4. **Delivery layer:** tables, nomograms, Excel workbook, GUI reports.

The theory is unified even though the software forms differ.

31. Historical and theoretical anchors

The project stands within a well-established line of nonlinear wave theory:

- Airy theory for infinitesimal waves,
- Stokes perturbation expansions for weakly nonlinear waves,

- cnoidal theory for long shallow-water waves,
- stream-function and Fourier methods for strongly nonlinear finite-depth waves.

The specific methodological lineage behind the main solver is associated with work by Dean, Rienecker, and especially John D. Fenton, whose papers and reviews developed the modern engineering use of fully nonlinear steady periodic wave computation (Rienecker and Fenton, 1981; Fenton and Rienecker, 1982; Fenton, 1988; Fenton, 1990; Fenton, 1999a; Fenton, 1999b).

32. Why the codebase mixes exact solvers and surrogates

This is not redundancy. It is an engineering design choice.

- The exact solver provides physical authority.
- The surrogates provide speed and portability.
- The workbook provides accessibility.
- The tables and nomograms provide design usability.

Without the exact solver, the surrogates would have no trustworthy target. Without the surrogates, the exact solver would be too heavy for many batch or spreadsheet workflows.

33. Practical notes on validity

The outputs are only as valid as the underlying steady-wave assumptions:

- two-dimensionality,
- periodicity,
- inviscid flow,
- irrotationality,
- no breaking,
- no surface tension,
- horizontal bed.

Very steep, breaking, strongly three-dimensional, or rapidly varying bathymetry cases lie outside the strict theory envelope.

34. Summary

The suite is best understood as a complete nonlinear wavelength and steady-wave-field ecosystem centered on the Fenton Fourier/stream-function solution of the exact free-surface problem. `fourier.cpp` is the clearest implementation of the core mathematics. `fenton_gui.py` is the main operational interface. `function.py` exposes the central nonlinear wavelength map. The remaining scripts and VBA modules either accelerate, tabulate, visualize, or operationalize that same physics.

At its heart, the entire project is built around one engineering truth:

$$L \neq L_{Linear-Airy}$$

once wave nonlinearity and current matter, and the physically consistent wavelength must be obtained from a nonlinear theory or from a surrogate trained against it.

35. Final engineering interpretation

For practical engineering use, the codebase should be read in the following order:

1. `fourier.cpp` for the core mathematics and solver architecture.
2. `fenton_gui.py` for the operational report workflow.
3. `function.py` for reusable scalar wavelength evaluation.
4. `tables.py` for tabulation.
5. `pade.py`, `neural.py`, `genetic.py` for fast surrogates.
6. VBA modules and workbook for deployment in Excel.
7. nomogram tools for graphical aids.

That order mirrors the hierarchy from physics to deployment.

Glossary of variables and symbols

A. Primary physical inputs and geometric quantities

- H — **wave height**, defined as crest elevation minus trough elevation. In engineering terms, it is the total vertical wave amplitude from trough to crest. It is a primary prescribed input across the suite and also enters multiple nondimensional groups such as H/d , H/L , and the Ursell number.
- T — **wave period**, i.e. the time required for one full wave cycle to pass a fixed point. It is one of the main prescribed inputs for wavelength prediction and is related to angular frequency by $\omega = 2\pi/T$. In the exact solver it may be prescribed while wavelength is solved for; in surrogate models it appears both directly and through nondimensional combinations such as $T\sqrt{g/d}$.
- d — **still-water depth**, measured vertically from the mean still-water level to the flat impermeable bed. It is the main depth scale in the project and is repeatedly used as the reference length in nondimensionalization.
- L — **wavelength**, i.e. crest-to-crest horizontal distance of the steady periodic wave. It is the central predicted quantity of the whole project.
- λ — **alternative symbol for wavelength** used in some closure equations and explanatory derivations. In this README it has the same physical meaning as L .
- k — **wavenumber**, defined by $k = \frac{2\pi}{L}$. It is the reciprocal spatial scale of the wave and appears naturally in all Fourier, dispersion, and collocation formulations.
- kd — **relative depth parameter**, defined by $kd = \frac{2\pi d}{L}$. It is one of the most important regime indicators in wave mechanics, distinguishing shallow-, intermediate-, and deep-water behavior.
- ω — **angular frequency**, defined by $\omega = \frac{2\pi}{T}$. It is the radian-frequency form of the wave period.

- τ — **dimensionless period variable** used in the depth-scaled formulation, $\tau = T\sqrt{\frac{g}{d}}$. In some parts of the documentation it is also used as a period symbol in classical wave-theory notation; the local meaning should therefore be read from context.
 - σ — **dimensionless angular frequency** associated with τ , commonly written as $\sigma = \frac{2\pi}{\tau}$. It appears in approximate dispersion initializers and theoretical nondimensional forms.
 - α — **squared dimensionless frequency parameter**, introduced as $\alpha = \sigma^2$. It is a compact helper variable in approximate dispersion relations.
 - c — **phase speed** or **celerity**, i.e. wave propagation speed, $c = \frac{L}{T}$. In period-specified nonlinear problems it is not known a priori and must be solved consistently with the free-surface equations.
 - g — **gravitational acceleration**, the standard body-force constant controlling wave dispersion and hydraulic scaling.
 - ρ — **fluid density**, used in energy, momentum, impulse, and radiation-stress expressions.
-

B. Coordinates, geometry, and free-surface description

- x — **horizontal spatial coordinate** in the direction of wave propagation.
 - y — **vertical coordinate**, taken positive upward.
 - t — **time**.
 - ξ — **travelling-wave phase coordinate**, $\xi = x - ct$, used to convert the unsteady physical problem into a steady problem in the moving frame.
 - $\eta(x, t)$ — **free-surface elevation** relative to the still-water level in the physical frame.
 - $\eta(\xi)$ — **steady free-surface profile** in the travelling frame.
 - $\eta(x)$ — shorthand for the free-surface elevation when the travelling-frame steady representation is already implied.
 - η_m — **collocated free-surface ordinate** at collocation point m . These values are the unknown surface elevations enforced in the nonlinear algebraic system.
 - η_{crest} — **crest elevation** above the still-water level.
 - η_{trough} — **trough elevation** below the still-water level.
 - η_c, η_t — compact output-style notation for **crest** and **trough elevations**.
 - $\bar{\eta}$ — **mean free-surface elevation over one wavelength**, used to diagnose or constrain set-up / set-down and mean-level normalization.
-

C. Velocity, potential, and stream-function variables

- ϕ — **velocity potential**. For irrotational flow, the velocity field is the gradient of ϕ .

- ψ — **stream function**. In the two-dimensional incompressible formulation it provides an equivalent harmonic representation of the flow and is especially convenient because both the bed and the free surface are streamlines in the moving frame.
 - u — **horizontal velocity component**.
 - v — **vertical velocity component** in the two-dimensional (x, y) formulation.
 - w — **vertical velocity component** in sections or code comments using (x, z) -style notation. It represents the same physical quantity as v , just under a different coordinate-label convention.
 - u_m — **horizontal velocity evaluated at collocation point m** on the free surface.
 - $\tilde{u}_m$ — **wave-induced part of the horizontal velocity** at collocation point m , separated from the travelling-frame mean contribution.
 - u_b — **instantaneous horizontal velocity at the bed**.
 - u_b^2 or ub^2 — **mean square orbital bed velocity**, defined in the output glossary as the phase-averaged square of the oscillatory bed velocity relative to the Eulerian mean current.
 - $u_{b,\text{rms}}$ or ub,rms — **root-mean-square orbital bed velocity**, $u_{b,\text{rms}} = \sqrt{u_b^2}$.
 - $u_{\text{surf,max}}$ or $usurf,\text{max}$ — **maximum horizontal velocity at the free surface**, obtained by scanning phase over one wave cycle.
 - $u_{\text{bed,max}}$ or $ubed,\text{max}$ — **maximum horizontal velocity at the bed**, obtained by scanning phase over one wave cycle.
 - $a_{x,\text{max}}$ or a_x,max — **maximum horizontal acceleration magnitude**, typically based on the convective or total derivative of the horizontal velocity.
-

D. Current definitions and flux-related variables

- U_c — **prescribed Eulerian ambient current velocity** used throughout the project. It is aligned with the direction of propagation and is the current variable appearing in linear, nonlinear, and surrogate formulations.
- $\bar{u}_1$ — **Eulerian mean current** in the exact-solver notation. It is related to the travelling-frame mean speed by $\bar{u}_1 = c - \bar{U}$.
- $\bar{u}_2$ — **Stokes / mass-transport current** or **discharge-related current**, defined in the solver documentation by $\bar{u}_2 = c - \frac{Q}{d}$.
- $\bar{U}$ — **travelling-frame mean speed**. This is a key exact-solver internal variable used to distinguish the wave frame from the prescribed Eulerian current.
- $\hat{U}_c$ — **depth-scaled nondimensional current**, $\hat{U}_c = \frac{U_c}{\sqrt{gd}}$.
- Q — **volume flux per unit width** or **depth-integrated discharge** in the moving frame. It is one of the most important integral invariants of the steady-wave problem and also acts as the streamline offset between free surface and bed depending on normalization.

- q — **wave volume flux correction / streamline-offset variable** appearing in the collocation residual structure. In the output glossary it is written as $q = \bar{U}d - Q$.
 - Fr_c — **current Froude number**, $Fr_c = \frac{U_c}{\sqrt{gd}}$, measuring current strength relative to the shallow-water gravity-wave speed.
 - D — **Doppler factor**, $D = \frac{U_c T}{L}$, used as a compact current-period-wavelength interaction measure.
-

E. Fourier / spectral representation variables

- N — **Fourier order or number of retained harmonics / collocation intervals**, depending on context. It controls spectral resolution.
 - n — **generic harmonic index** in Fourier series. It usually runs from 1 to N .
 - m — **collocation-point index**. It usually runs from 0 to N .
 - a_n — **Fourier coefficients of the free-surface elevation** in the cosine-series representation $\eta(x) = \sum_{n=0}^N a_n \cos(nkx)$.
 - a_0 — **mean-level correction term** in the free-surface cosine series.
 - a_1 — **fundamental free-surface harmonic coefficient**.
 - a_2 — **first nonlinear correction harmonic** of the free surface.
 - b_n — **stream-function Fourier coefficients** in the theoretical stream-function representation.
 - B — **constant background term** in the stream-function representation $\psi(x, y) = By + \sum_{n=1}^N \dots$. It represents the linear-in- y part of the stream function.
 - B_j — **implementation-level Fourier / harmonic coefficients** of the nonlinear solver state vector. In the README these are the exact-solver coefficients stored in the unknown vector and used in collocation residual assembly.
-

F. Collocation, solver state, and residual notation

- x_m — **physical-space collocation coordinate** used in the general theoretical presentation, $x_m = \frac{m\pi}{Nk}$.
- X_m — **phase-based collocation coordinate** used in the implementation-oriented nondimensional presentation, $X_m = \frac{m\pi}{N}$. It represents the same collocation logic but in a shifted / scaled phase coordinate.
- $\mathbf{z}$ — **global nonlinear unknown vector** in Newton iteration. It contains all unknowns of the steady-wave problem, such as wavelength or wavenumber, surface ordinates, harmonic coefficients, current-related quantities, and Bernoulli constant.
- $\mathbf{z}^{(m)}$ — **current Newton iterate** at iteration m .
- $\Delta \mathbf{z}^{(m)}$ — **Newton correction vector** applied to update the current iterate.

- $F(z)$ — **nonlinear residual vector** of the full steady-wave system. The exact solution satisfies $F(z) = 0$.
 - J — **Jacobian matrix** of the nonlinear residual system.
 - J_{ij} — **Jacobian entry**, $J_{ij} = \frac{\partial F_i}{\partial z_j}$.
 - $R_m^{(\psi)}$ — **streamline residual** at collocation point m , $R_m^{(\psi)} = \psi(X_m, \eta_m) + q$. It enforces the fact that the free surface is a streamline.
 - $R_m^{(B)}$ — **Bernoulli residual** at collocation point m , enforcing the dynamic free-surface condition in collocated form.
 - z_7 — exact-solver state component associated in the README with $z_7 \leftrightarrow \bar{U} \sqrt{\frac{k}{g}}$.
 - z_8 — exact-solver state component associated with $z_8 \leftrightarrow \bar{U} kd - q$.
 - z_9 — exact-solver state component associated with the Bernoulli constant R .
 - z_{10+m} — exact-solver state entries storing the collocated free-surface values $k\eta_m$.
-

G. Governing-equation and free-surface constants

- R — **Bernoulli constant** in the moving-frame steady free-surface condition. It plays the role of a total-head constant and is one of the key scalar unknowns of the nonlinear wave problem.
 - r — **reduced Bernoulli constant**, defined in the output glossary as $r = R - gd$. It is a shifted form of R used for interpretation and reporting.
 - $B(t)$ — **time-dependent Bernoulli integration function** appearing in the unsteady potential-flow Bernoulli equation before transforming to the steady moving frame.
 - $C(t)$ — **time-only integration function** in the unsteady Bernoulli equation, removable by a redefinition of the potential in standard irrotational-flow theory.
-

H. Integral hydraulic quantities and engineering outputs

- E — **total wave energy density** per unit horizontal area. In the exact problem it is the sum of potential and kinetic contributions.
- E_p — **potential energy density**.
- E_k — **kinetic energy density**.
- T (*hydraulic-output meaning*) — **kinetic energy density** in the project output glossary. This symbol is overloaded with the wave period T ; the surrounding section must therefore be read carefully.
- V — **potential energy density** in the project output glossary.
- I — **wave impulse per unit width** or **wave momentum integral**, used as an integral invariant and also in Stokes-drift-related diagnostics.

- S — **momentum flux** in the moving frame.
 - S_{xx} — **radiation stress component in the direction of propagation**.
 - F (*hydraulic-output meaning*) — **wave power / energy flux**. This symbol is distinct from the nonlinear residual vector $F(z)$ and from the scalar linear-dispersion residual $F(k)$.
 - C_g — **group velocity**, defined in the project output glossary by $C_g = \frac{F}{E}$.
 - PE — shorthand used in some code comments for **potential energy**.
 - KE — shorthand used in some code comments for **kinetic energy**.
 - P — shorthand used in code comments for **power / energy flux**.
-

I. Stability, regime, and classification variables

- Ur or U_r — **Ursell number**, $Ur = \frac{HL^2}{d^3}$, a standard measure of the relative importance of nonlinearity and dispersion.
 - $H_{\max}$ or $H_{\max}$ — **Miche limiting wave height**, used as an engineering breaking / stability diagnostic.
 - **Saturation — breaking index** defined conceptually as $\frac{H}{H_{\max}}$. Values above unity indicate instability / breaking in the simple Miche-limit interpretation.
 - **Regime — depth-regime classification**, typically based on d/L , separating shallow, intermediate, and deep-water conditions.
 - **Asymmetry — velocity asymmetry measure**, defined in the project output glossary as a crest-to-trough horizontal-velocity ratio such as $|u_{\text{crest}}|/|u_{\text{trough}}|$.
-

J. Nondimensional variables used throughout the suite

- $\hat{H}$ — **depth-scaled nondimensional wave height**, $\hat{H} = \frac{H}{d}$.
 - $\hat{L}$ — **depth-scaled nondimensional wavelength**, $\hat{L} = \frac{L}{d}$.
 - $\hat{c}$ — **depth-scaled nondimensional celerity**, $\hat{c} = \frac{c}{\sqrt{gd}}$.
 - $\frac{H}{L}$ — **wave steepness**.
 - $\frac{d}{L}$ — **relative depth**.
 - $\frac{H}{d}$ — **relative wave height**.
 - $\frac{\lambda}{d}$ — **relative wavelength**, used in wavelength-specified closure relations.
-

K. Linear-dispersion and baseline-solver notation

- $F(k)$ — **scalar residual of the Doppler-shifted linear dispersion relation** in `linear.bas`, $F(k) = \left(\frac{2\pi}{T} - kU_c\right)^2 - gk \tanh(kd)$. Its root defines the baseline wavenumber.
 - $F'(k)$ — **derivative of the scalar dispersion residual**, used in Newton iteration for the baseline linear solver.
 - k_n — **Newton iterate of the wavenumber** in the scalar linear solver.
 - k_{n+1} — **updated wavenumber iterate** after one Newton step.
 - $\frac{1}{\cosh(kd)}$ — **hyperbolic secant** appearing in the derivative of the Doppler-shifted dispersion residual.
-

L. Surrogate-model notation

L.1 General surrogate symbols

- L_{lin} — **baseline linear wavelength**, used as the reference wavelength in surrogate formulas.
- L_{base} — **generic baseline wavelength** symbol used in rational / Padé surrogate descriptions.
- $\mathcal{L}_{\text{lin}}(\cdot, \cdot)$ — **linear finite-depth wavelength operator**, i.e. the mapping that returns wavelength by solving the linear dispersion relation for the supplied effective period and depth.

L.2 Genetic symbolic-regression formula

- L_{genetic} — **wavelength predicted by the project-specific genetic symbolic-regression formula**.
- M — **multiplicative correction factor** applied to the baseline linear wavelength in the genetic formula.
- t_1, t_2, t_3 — **three explicit sub-terms** making up the genetic multiplicative correction M . They respectively represent an exponential current-period effect, a bounded rational current term, and a hyperbolic-tangent compensation term involving relative height and nondimensional period.

L.3 Effective transformed-variable formula

- T_{eff} — **effective transformed period** in the project-specific effective formula. It is not a measured period; it is an engineered surrogate variable passed into a linear dispersion solve.
- d_{eff} — **effective transformed depth** in the effective formula. Like T_{eff} , it is an apparent or transformed variable rather than a direct physical measurement.
- Δ_T — **additive correction applied to the physical period** such that $T_{\text{eff}} = T + \Delta_T$.
- Δ_d — **additive correction applied to the physical depth** such that $d_{\text{eff}} = d + \Delta_d$.

- $L_{\text{effective}}$ — **wavelength predicted by the project-specific effective transformed-variable formula**, obtained by solving the linear dispersion relation with T_{eff} and d_{eff} .

L.4 Padé / rational surrogate notation

- $p_0, p_1, p_2, \dots$ — **numerator coefficients** in the rational Padé-style surrogate representation.
- $q_1, q_2, \dots$ — **denominator coefficients** in the rational Padé-style surrogate representation.
- $x_1, x_2, \dots$ — **generic surrogate feature variables** entering the rational approximation. These stand for transformed combinations of the physical inputs, not for the physical horizontal coordinate.

M. Matrix-factorization and linear-algebra notation

- U — **left singular-vector matrix** in the SVD factorization of the Jacobian.
- Σ — **diagonal matrix of singular values** in the SVD factorization.
- V^T — **transpose of the right singular-vector matrix** in the SVD factorization.
- Σ^{-1} — **inverse or truncated inverse of the singular-value matrix**, used to build the pseudoinverse correction.

N. Practical note on overloaded symbols

Some symbols are intentionally **overloaded** across different parts of the README:

- T can mean **wave period** or, in the hydraulic-output glossary style, **kinetic energy density**.
- F can mean the **nonlinear residual vector** $F(z)$, the **scalar dispersion residual** $F(k)$, or **wave power / energy flux**.
- R can mean the **Bernoulli constant**; residuals are therefore written explicitly as $R_m^{(\psi)}$ and $R_m^{(B)}$ to avoid confusion.
- v and w both denote the **vertical velocity component**, depending on whether the notation follows (x, y) or (x, z) -style conventions.

In all such cases, the **local section context governs the intended meaning**.

Bibliography

1. **Fenton, J. D. (1999).** *Numerical methods for nonlinear waves*. In P. L.-F. Liu (ed.), *Advances in Coastal and Ocean Engineering*, Vol. 5, World Scientific, Singapore, pp. 241–324.

Relevance: Review of fully nonlinear wave computation methods (Fourier, BIE, polynomial approximation).

URL: [Fenton99Liu-Numerical-methods-for-nonlinear-waves.pdf](#)

2. **Fenton, J. D. (1999).** *The cnoidal theory of water waves.* In J. B. Herbich (ed.), *Developments in Offshore Engineering*, Gulf Publishing, Houston, pp. 275–337.
Relevance: Cnoidal-wave theory (finite-depth, long waves) and numerical formulations.
URL: [Fenton99Cnoidal-The-cnoidal-theory-of-water-waves.pdf](#)
3. **Fenton, J. D. & Kennedy, A. B. (1996).** *Fast methods for computing the shoaling of nonlinear waves.* In *Proceedings of the 25th International Conference on Coastal Engineering*, Vol. 1, Orlando, pp. 1130–1143.
Relevance: Nonlinear wave propagation and shoaling over varying bathymetry.
URL: [Fenton96+Kennedy-Fast-methods-for-computing-the-shoaling-of-nonlinear-waves.pdf](#)
4. **Fenton, J. D. (1995).** *A numerical cnoidal theory for steady water waves.* In *Proceedings of the 12th Australasian Coastal and Ocean Engineering Conference*, Melbourne, pp. 175–180.
Relevance: Cnoidal-wave theory (finite-depth, long waves) and numerical formulations.
URL: [Fenton95-A-numerical-cnoidal-theory-for-steady-water-waves.pdf](#)
5. **Townsend, M. & Fenton, J. D. (1995).** *Numerical comparisons of wave analysis methods.* In *Proceedings of the 12th Australasian Coastal and Ocean Engineering Conference*, Melbourne, pp. 169–173.
Relevance: Wave analysis and inversion from pressure measurements; conditioning and method comparison.
URL: [Townsend95+Fenton-Numerical-comparisons-of-wave-analysis-methods.pdf](#)
6. **Kennedy, A. B. & Fenton, J. D. (1995).** *Simulation of the propagation of surface gravity waves using local polynomial approximation.* In *Proceedings of the 12th Australasian Coastal and Ocean Engineering Conference*, Melbourne, pp. 287–292.
Relevance: Nonlinear wave propagation and shoaling over varying bathymetry.
URL: [Kennedy95+Fenton-Simulation-of-the-propagation-of-surface-gravity-waves-using-local-polynomial-approximation.pdf](#)
7. **Fenton, J. D. (1993).** *Simulating wave shoaling with boundary integral equations.* In *Proceedings of the 11th Australasian Conference on Coastal and Ocean Engineering*, Townsville, pp. 71–76.
Relevance: Boundary-integral formulation for nonlinear wave transformation, with singularity subtraction.
URL: [Fenton93-Simulating-wave-shoaling-with-boundary-integral-equations.pdf](#)
8. **Fenton, J. D. (1990).** *Nonlinear wave theories.* In B. Le Méhauté & D. M. Hanes (eds.), *The Sea: Ocean Engineering Science, Part A*, Vol. 9, Wiley, New York, pp. 3–25.
Relevance: Survey and selection guidance: Stokes, cnoidal, stream-function, and related theories.
URL: [Fenton90b-Nonlinear-wave-theories.pdf](#)
9. **Drennan, W. M., Fenton, J. D. & Donelan, M. A. (1990).** *Numerical simulation of nonlinear wave groups.* In *Proceedings of the 11th Annual Conference of the Canadian Applied Mathematics Society*, Halifax.
Relevance: Relevant to nonlinear and free-surface wave modelling.
URL: [Drennan90-Numerical-simulation-of-nonlinear-wave-groups.pdf](#)
10. **Fenton, J. D. & McKee, W. D. (1990).** *On calculating the lengths of water waves.*

Coastal Engineering, **14**, 499–513.

Relevance: Wavelength determination for nonlinear waves in finite depth.

URL: [Fenton90c+McKee-On-calculating-the-lengths-of-water-waves.pdf](#)

11. **Fenton, J. D. (1988).** *The numerical solution of steady water wave problems. Computers and Geosciences*, **14**, 357–368.
Relevance: Core stream-function / Fourier algorithm for steady, periodic nonlinear waves.
URL: [Fenton88-The-numerical-solution-of-steady-water-wave-problems.pdf](#)
12. **Fenton, J. D. (1986).** *Polynomial approximation and water waves.* In *Proceedings of the 20th International Conference on Coastal Engineering*, Vol. 1, Taipei, pp. 193–207.
Relevance: Relevant to nonlinear and free-surface wave modelling.
URL: [Fenton86-Polynomial-approximation-and-water-waves.pdf](#)
13. **Fenton, J. D. (1985).** *A fifth-order Stokes theory for steady waves. Journal of Waterway, Port, Coastal, and Ocean Engineering*, **111**, 216–234.
Relevance: Closed-form fifth-order Stokes expansion for steady waves in finite depth.
URL: [Fenton85d-A-fifth-order-Stokes-theory-for-steady-waves.pdf](#)
14. **Fenton, J. D. (1983).** *On the application of steady wave theories.* In *Proceedings of the 6th Australasian Conference on Coastal and Ocean Engineering*, Christchurch, pp. 65–70.
Relevance: Guidance on applicability and limits of steady-wave theories in engineering.
URL: [Fenton83-On-the-application-of-steady-wave-theories.pdf](#)
15. **Fenton, J. D. & Rienecker, M. M. (1982).** *A Fourier method for solving nonlinear water wave problems. Journal of Fluid Mechanics*, **118**, 411–443.
Relevance: Fourier-series method for solving fully nonlinear steady water waves.
URL: [Fenton82c+Rienecker-A-Fourier-method-for-solving-nonlinear-water-wave-problems.pdf](#)
16. **Schwartz, L. W. & Fenton, J. D. (1982).** *Strongly-nonlinear waves.* In M. Van Dyke, J. V. Wehausen & J. L. Lumley (eds.), *Annual Review of Fluid Mechanics*, **14**, 39–60.
Relevance: Fundamental properties and approximations for strongly nonlinear wave motion.
URL: [Schwartz82-Strongly-nonlinear-waves.pdf](#)
17. **Rienecker, M. M. & Fenton, J. D. (1981).** *A Fourier approximation method for steady water waves. Journal of Fluid Mechanics*, **104**, 119–137.
Relevance: Fourier-series method for solving fully nonlinear steady water waves.
URL: [Rienecker81+Fenton-A-Fourier-approximation-method-for-steady-water-waves.pdf](#)
18. **Fenton, J. D. & Rienecker, M. M. (1980).** *Accurate numerical solutions for nonlinear waves.* In *Proceedings of the 17th International Conference on Coastal Engineering*, Sydney, pp. 50–69.
Relevance: Benchmark accurate numerical solutions for nonlinear wave profiles and kinematics.
URL: [Fenton80+Rienecker-Accurate-numerical-solutions-for-nonlinear-waves.pdf](#)
19. **Fenton, J. D. (1979).** *A high-order cnoidal wave theory. Journal of Fluid Mechanics*, **94**, 129–161.
Relevance: Cnoidal-wave theory for finite-depth long waves and associated numerical

formulations.

URL: [Fenton79-A-high-order-cnoidal-wave-theory.pdf](#)

20. **Fenton, J. D. & Mills, D. A. (1976).** *Shoaling waves: numerical solution of exact equations*. In D. G. Provis & R. Radok (eds.), *Proceedings of the IUTAM Symposium on Waves on Water of Variable Depth*, Canberra, Springer-Verlag, pp. 93–100.

Relevance: Nonlinear wave propagation and shoaling over varying bathymetry.

URL: [Fenton76+Mills-Shoaling-waves-Numerical-solution-of-exact-equations.pdf](#)

21. **Fenton, J. D. (1972).** *A ninth-order solution for the solitary wave*. *Journal of Fluid Mechanics*, **53**, 257–271.

Relevance: High-order analytic and series solution for solitary waves.

URL: [Fenton72-A-ninth-order-solution-for-the-solitary-wave.pdf](#)